

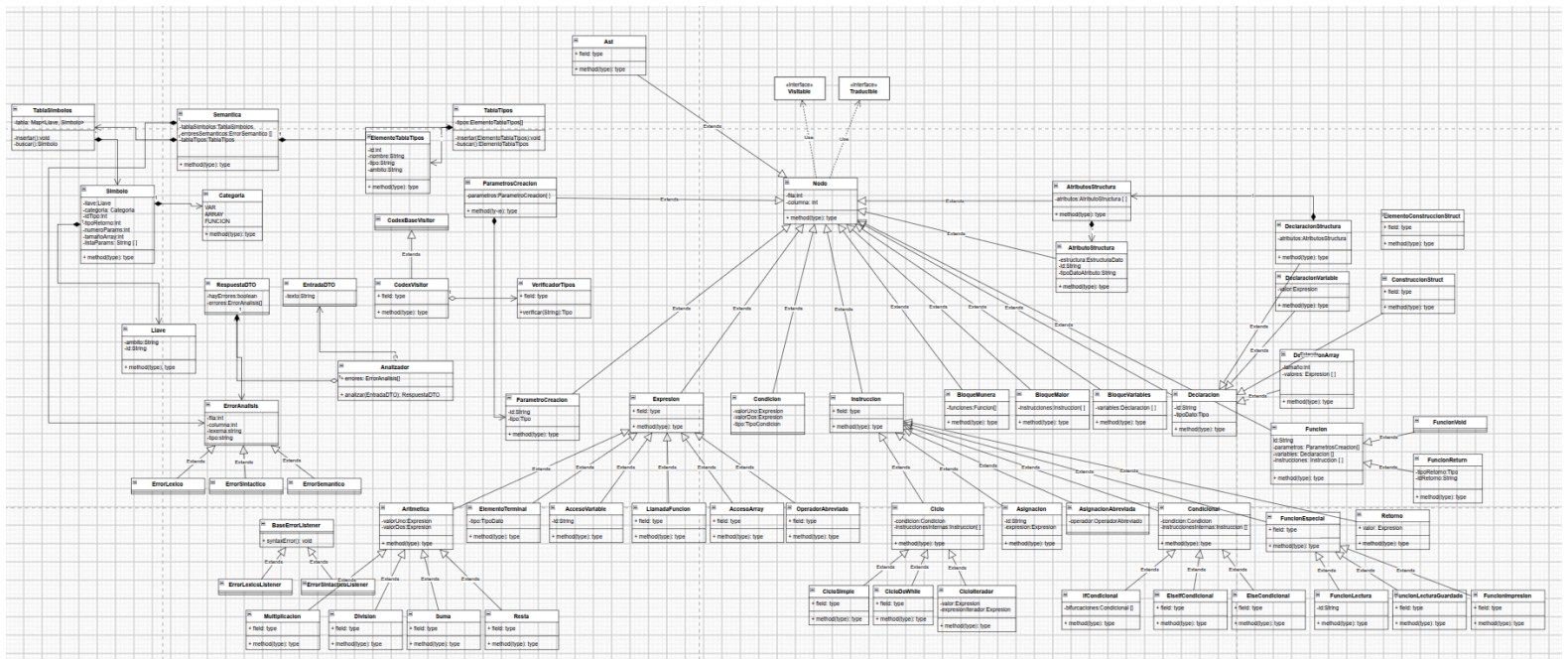
MANUAL TECNICO

Practica Uno Compiladores 2 : Codex Latinus
Version 1.0

Desarrollador: Ronaldo Hilario.

Frontend: Angular version 17.
Backend: Java version 21.
Generador Lexer Y Parser: Antlr4.
Visualizacion de Ast: Graphviz.
Protocolo de Comunicación: API REST

Protocolo de Comunicación: API REST



GRAMATICA

grammar Codex;

/** Analisis Sintactico **/

inicio:

 bloque_variables bloque_munera bloque_maior
 ;

/** BLOQUE VARIABLES **/

bloque_variables:

 VARIABLES MAYOR_Q variable*
 | /** lambda **/
 ;

variable:

 variable_simple
 | variable_compuesta
 | dec_structura
 | constr_structura
 ;

/** TIPO DE DATOS SIMPLES Y CADENAS **/

variable_simple

 : dec_var P_COMA
 ;

dec_var:

 ESTO ID DOS_P tipo_dato expresion

;

/**TIPO DE DATOS COMPUESTOS (ARRAYS)**/

variable_compuesta

: dec_array

;

dec_array

: SERIES ID CORCH_A ENTERO CORCH_C DOS_P tipo_dato
ini_array P_COMA

;

tipo_dato

: NUMERUS

| DECIMALIS

| TEXTUM

| LITTERA

| BOOL

;

ini_array

: LLAVE_A valores_ini_array LLAVE_C

| /**lambda**/

;

valores_ini_array

: valores_ini_array COMA expresion

| expresion

;

/**ESTRUCTURAS**/

/**Declaracion**/

dec_structura

 : STRUCTURA ID_STRUCT LLAVE_A atributos LLAVE_C FINIS
P_COMA
 ;

atributos

 : atributos separador atributo
 | atributo
 ;

separador

 : COMA
 | P_COMA
 | /**lambda**/
 ;

atributo

 : tipo ID DOS_P tipo_dato_atributo separador
 ;

tipo

 : ESTO
 | SERIES
 ;

tipo_dato_atributo

 : tipo_dato
 | ID_STRUCT
 | ID
 ;

/**construccion de la estructura**/

constr_structura

: ESTO ID DOS_P ID_STRUCT LLAVE_A elementos_construccion
LLAVE_C
;

elementos_construccion

: elementos_construccion COMA ele_construccion
| ele_construccion
;

ele_construccion

: ID DOS_P valor_elem_construccion tamaño_array
;

valor_elem_construccion

: expresion
| ID_STRUCT tamaño_array
;

tamaño_array

: CORCH_A ENTERO CORCH_C
| /**lambda**/
;

/**asignacion a una posicion en especifico a un atributo de estructura de tipo
array**/

asig_atributo_array

: expresion tamaño_array EQ LLAVE_A LLAVE_C
;

```
asig_atributo_struct
    : llamada_atributo_struct EQ expresion P_COMA
    ;
```

```
elementos_asig
    : elementos_asig COMA ID DOS_P expresion
    | ID DOS_P expresion
    ;
```

/**ASIGNACION A VARIABLES SIMPLES**/

```
asignacion
    : ID EQ expresion P_COMA
    ;
```

/**ASIGNACION A ARREGLOS**/

```
asignacion_array
    : ID tamaño_array EQ expresion P_COMA
    ;
```

/**BLOQUE MUNERA**/

```
bloque_munera:
    MUNERA funcion*
    | /**lambda**/
    ;
```

/**funciones**/

```
funcion
    : funcion_sin_retorno
```

```
| funcion_con_retorno  
;
```

```
/** FUNCIONES **/
```

```
funcion_sin_retorno
```

```
: ACTIO ID
```

```
PAR_A parametros PAR_C
```

```
LLAVE_A seccion_var_funcion? instruccion*
```

```
LLAVE_C FINIS
```

```
P_COMA
```

```
;
```

```
funcion_con_retorno
```

```
: RATIO (tipo_dato | ID_STRUCT) ID
```

```
PAR_A parametros PAR_C
```

```
LLAVE_A seccion_var_funcion? instruccion*
```

```
LLAVE_C FINIS
```

```
P_COMA
```

```
;
```

```
seccion_var_funcion
```

```
: VARIABLES CORCH_A variable* CORCH_C P_COMA?
```

```
;
```

```
tipo_retorno_funcion
```

```
: tipo_dato
```

```
| ID_STRUCT
```

```
;
```



```

//lectura
fun_lectura
    : MENOR_Q MENOR_Q
    ;

fun_lectura_guardado
    : ID MENOR_Q MENOR_Q separador
    ;

//impresion
fun_impresion
    : impresion P_COMA
    ;

impresion
    : impresion MAYOR_Q MAYOR_Q expresion
    | MAYOR_Q MAYOR_Q expresion
    ;

parametros
    : parametros COMA parametro
    | parametro
    | /**lambda**/
    ;

parametro
    : ESTO ID DOS_P tipo_dato
    ;

/**BLOQUE MAIOR**/

```

bloque_maior
: MAIOR instruccion* FINIS_MAY P_COMA
;

retorno
: REDDERE expresion P_COMA
;

instruccion
: condicional
| ciclo_simple
| ciclo_do_while
| ciclo_iterador
| operacion_abrev P_COMA
| asignacion
| asignacion_array
| fun_lectura
| fun_lectura_guardado
| fun_impresion
| llamada_atributo_struct
| asig_atributo_struct
| llamada_funcion P_COMA
| retorno
| PERGE P_COMA
| INTERRUPE P_COMA
;
/**CONDICIONALES**/

condicional

```
        : SI PAR_A condicion PAR_C LLAVE_A instruccion* LLAVE_C
mas_condicionales* FINIS P_COMA
;
```

mas_condicionales

```
        : ALITER PAR_A condicion PAR_C LLAVE_A instruccion* LLAVE_C
mas_condicionales*
| ALITER LLAVE_A instruccion* LLAVE_C
;
```

/**CICLOS**/

ciclo_simple

```
        : DUM PAR_A condicion PAR_C LLAVE_A instruccion* LLAVE_C
FINIS P_COMA
;
```

ciclo_do_while

```
        : FACERE LLAVE_A instruccion* LLAVE_C DUM PAR_A condicion
PAR_C P_COMA
;
```

ciclo_iterador

```
        : PER
PAR_A
dec_var P_COMA
condicion P_COMA
expresion_iterador
PAR_C LLAVE_A instruccion*
LLAVE_C
;
```

expresion_iterador
: expresion
| operacion_abrev
;

operacion_abrev
: ID MAS_MAS
| ID MENOS_MENOS
;

llamada_atributo_struct
:expresion
;

expresion
: PAR_A expresion PAR_C
| expresion MULTI expresion
| expresion DIV expresion
| expresion MAS expresion
| expresion MENOS expresion
| valor_posicion_array
| llamada_funcion
| expresion PUNTO ID
| ENTERO
| DECIMAL
| CADENA
| CHAR
| VERUM
| FALSUS
| ID
;

valor_posicion_array
: ID CORCH_A ENTERO CORCH_C
;

llamada_funcion
: ID PAR_A parametros_llamada PAR_C
;

parametros_llamada
: parametros_llamada COMA expresion
| expresion
| /**lambda**/
;

condicion
: NON condicion
| expresion EQ_EQ expresion
| expresion NO_EQ expresion
| expresion MAYOR_EQ_Q expresion
| expresion MAYOR_Q expresion
| expresion MENOR_EQ_Q expresion
| expresion MENOR_Q expresion
| condicion AND condicion
| condicion OR condicion
| PAR_A condicion PAR_C
| expresion
;

/**Análisis Lexico**/

/**palabras reservadas**/

VARIABLES: 'VARIABLES';
MUNERA: 'MUNERA>';
MAIOR: 'MAIOR>';
ESTO: 'esto';
SERIES: 'series';
NUMERUS: 'numerus';
DECIMALIS: 'decimalis';
TEXTUM: 'textum';
LITTERA: 'littera';
BOOL: 'bool';
VERUM: 'verum';
FALSUS: 'falsus';
STRUCTURA: 'structura';
SI: 'si';
FINIS: 'finis';
FINIS_MAY: 'FINIS';
ALITER: 'aliter';
DUM: 'dum';
FACERE: 'facere';
PER: 'per';
PERGE: 'perge';
INTERRUMPE: 'interrumpe';
ACTIO: 'actio';
RATIO: 'ratio';
REDDERE: 'reddere';
NON: 'non';

/**simbolos**/

MAS: '+';
MENOS: '-';
MULTI: '*';
DIV: '/';

EQ: '=';
EQ_EQ: '==';
NO_EQ: '!=';
MAYOR_Q: '>';
MAYOR_EQ_Q: '>=';
MENOR_Q: '<';
MENOR_EQ_Q: '<=';
AND: '&&';
OR: '||';
MAS_MAS: '++';
MENOS_MENOS: '--';
PUNTO: '.';
COMA: ',';
DOS_P: '::';
P_COMA: ';;';
LLAVE_A: '{';
LLAVE_C: '}';
CORCH_A: '[';
CORCH_C: ']';
PAR_A: '(';
PAR_C: ')';

//expresiones regulares

ID_STRUCT: [A-Z][a-zA-Z0-9_]* ;
ID: [a-zA-Z_][a-zA-Z0-9_]* ;
ENTERO: [0-9]+ ;
DECIMAL: [0-9]+'.'[0-9]+;
CADENA : "" .*? "" ;
CHAR : \" . \" ;
COMENTARIO_LINEA : '/' ~[\\r\\n]* ->skip;
COMENTARIO_BLOQUE : '##' .*? '##' -> skip ;
WS: [\\t\\n\\r\\f\\u00A0\\u200B] -> skip;

TABLA COMPATIBILIDAD

Contatenacion (+)

	BOOL	LITTERA	NUMERUS	DECIMALIS	TEXTUM
BOOL	error	error	error	error	textum
LITTERA	error	numerus	numerus	decimalis	textum
NUMERUS	error	numerus	numerus	decimalis	textum
DECIMALIS	error	decimalis	decimalis	decimalis	textum
TEXTUM	textum	textum	textum	textum	textum

Otras Operaciones(-, *, /)

	BOOL	LITTERA	NUMERUS	DECIMALIS	TEXTUM
BOOL	error	error	error	error	error
LITTERA	error	numerus	numerus	decimalis	error
NUMERUS	error	numerus	numerus	decimalis	error
DECIMALIS	error	decimalis	decimalis	decimalis	error
TEXTUM	error	error	error	error	error